

A câmera se move com as teclas W, A, S, D (para avançar, retroceder e se deslocar lateralmente) e gira com as setas (para rotacionar). A altura da câmera é fixa para simular uma caminhada natural. O labirinto é composto por paredes feitas de cubos e há uma verificação básica de colisão para evitar que a câmera atravessasse os obstáculos.

Implementação:

- **Movimentação da Câmera:**
A posição da câmera (camX e camZ) e seu ângulo de rotação (camAngle) são atualizados conforme o usuário pressiona as teclas. A função `gluLookAt()` define a visão da câmera mantendo a mesma altura (camY).
- **Criação do Labirinto:**
Um conjunto fixo de paredes é definido em um array. Cada parede é desenhada usando `glutSolidCube()`, posicionada com `glTranslatef()` para formar caminhos abertos.
- **Colisão Simples:**
Antes de atualizar a posição da câmera, uma função de verificação compara a nova posição com a área de cada parede. Se a nova posição estiver muito próxima de algum cubo (usando um limiar simples), a movimentação é bloqueada.
- **Ambiente:**
Um chão (desenhado com um quadrilátero) serve como base para o labirinto, facilitando a percepção do espaço.

Resultados:

A execução do programa resulta em um ambiente 3D onde o usuário pode caminhar pelo labirinto utilizando os controles do teclado. A lógica de colisão impede que a câmera atravessasse as paredes, garantindo que o usuário explore apenas os caminhos abertos. Essa implementação integra conceitos de sistemas de coordenadas, movimentação de câmera e hierarquia de objetos, fundamentais para aplicações interativas em gráficos 3D.

```
C/C++  
  
// labirinto.c  
#include <GL/glut.h>  
#include <stdlib.h>  
#include <math.h>  
#include <stdio.h>  
  
// Estrutura para representar uma parede (cubo) no labirinto
```

```

typedef struct {
    float x, z; // Posição no plano XZ (a altura Y é fixa, pois os cubos
                // têm altura 1.0)
} Wall;

// Definição de algumas paredes para formar um labirinto simples
#define NUM_WALLS 8
Wall walls[NUM_WALLS] = {
    { -2.0, -3.0 },
    {  0.0, -3.0 },
    {  2.0, -3.0 },
    { -2.0, -1.0 },
    {  2.0, -1.0 },
    { -2.0,  1.0 },
    {  0.0,  1.0 },
    {  2.0,  1.0 }
};

// Variáveis de posição e orientação da câmera
float camX = 0.0, camY = 1.0, camZ = 5.0; // camY fixa para manter a mesma
altura
float camAngle = 0.0; // Ângulo de rotação da câmera (em graus)

// Velocidade de movimento e rotação
float moveSpeed = 0.2;
float turnSpeed = 3.0;

// Função para verificar colisão: retorna 1 se a nova posição (newX, newZ)
// estiver muito próxima de alguma parede; caso contrário, retorna 0.
int checkCollision(float newX, float newZ) {
    for (int i = 0; i < NUM_WALLS; i++) {
        // Considera cada parede como um cubo de lado 1.0 com centro em
        (walls[i].x, walls[i].z)
        float dx = newX - walls[i].x;
        float dz = newZ - walls[i].z;
        // Se a distância em X e Z for menor que um limiar, há colisão
        if (fabs(dx) < 0.7 && fabs(dz) < 0.7)
            return 1; // colisão detectada
    }
    return 0;
}

// Desenha o chão (um grande quadrado) para facilitar a visualização do
ambiente
void drawFloor() {
    glColor3f(0.6, 0.6, 0.6); // cor cinza
    glBegin(GL_QUADS);
        glVertex3f(-10.0, 0.0, -10.0);

```

```

        glVertex3f( 10.0, 0.0, -10.0);
        glVertex3f( 10.0, 0.0,  10.0);
        glVertex3f(-10.0, 0.0,  10.0);
    glEnd();
}

// Desenha as paredes do labirinto (cada parede é um cubo com lado 1.0)
void drawLabyrinth() {
    for (int i = 0; i < NUM_WALLS; i++) {
        glPushMatrix();
        // Eleva o cubo em 0.5 para que o piso toque sua base
        glTranslatef(walls[i].x, 0.5, walls[i].z);
        glColor3f(1.0, 0.0, 0.0); // cor vermelha para as paredes
        glutSolidCube(1.0);
        glPopMatrix();
    }
}

// Função de exibição
void display() {
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glLoadIdentity();

    // Calcula o ponto que a câmera olha (mantendo a mesma altura)
    float rad = camAngle * M_PI / 180.0;
    float camLookX = camX + sin(rad);
    float camLookZ = camZ - cos(rad);

    // Configura a câmera
    gluLookAt(camX, camY, camZ,
              camLookX, camY, camLookZ,
              0.0, 1.0, 0.0);

    // Desenha o chão e o labirinto
    drawFloor();
    drawLabyrinth();

    glutSwapBuffers();
}

// Função para capturar eventos do teclado para movimentação
void keyboard(unsigned char key, int x, int y) {
    float rad = camAngle * M_PI / 180.0;
    float newX = camX;
    float newZ = camZ;

    switch (key) {
        case 'w': // Avança para frente

```

```

        newX += sin(rad) * moveSpeed;
        newZ -= cos(rad) * moveSpeed;
        break;
    case 's': // Retrocede
        newX -= sin(rad) * moveSpeed;
        newZ += cos(rad) * moveSpeed;
        break;
    case 'a': // Move para a esquerda
        newX -= cos(rad) * moveSpeed;
        newZ -= sin(rad) * moveSpeed;
        break;
    case 'd': // Move para a direita
        newX += cos(rad) * moveSpeed;
        newZ += sin(rad) * moveSpeed;
        break;
    case 27: // ESC para sair
        exit(0);
}
// Verifica colisão antes de atualizar a posição
if (!checkCollision(newX, newZ)) {
    camX = newX;
    camZ = newZ;
}
glutPostRedisplay();
}

// Função para capturar teclas especiais (setas) para rotacionar a câmera
void specialKeys(int key, int x, int y) {
    switch (key) {
        case GLUT_KEY_LEFT:
            camAngle -= turnSpeed;
            break;
        case GLUT_KEY_RIGHT:
            camAngle += turnSpeed;
            break;
    }
    glutPostRedisplay();
}

// Inicializa o OpenGL e configura a projeção
void init() {
    glEnable(GL_DEPTH_TEST);
    glClearColor(0.0, 0.0, 0.0, 1.0);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluPerspective(60.0, 1.0, 0.1, 100.0);
    glMatrixMode(GL_MODELVIEW);
}

```

```
// Função principal
int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
    glutInitWindowSize(800, 600);
    glutCreateWindow("Labirinto 3D - Movimentação com Câmera");

    init();
    glutDisplayFunc(display);
    glutKeyboardFunc(keyboard);
    glutSpecialFunc(specialKeys);

    glutMainLoop();
    return 0;
}
```